

DESIGN PATTERNS

Design patterns selected

1. **Singleton** — AirportControlTower

An airport has exactly one control tower coordinating all flight operations. A Singleton ensures a single, globally-accessible instance manages runway assignments, departure clearances, and communication channels, preventing conflicting instructions from multiple instances.

2. **Observer** — FlightStatusNotifier

Many stakeholders (passengers, gate agents, baggage handlers, airlines) need real-time updates when a flight's status changes (delayed, boarding, cancelled). The Observer pattern decouples the flight entity from its subscribers, so new observers can be added without modifying the flight class.

3. **Strategy** — CheckInStrategy

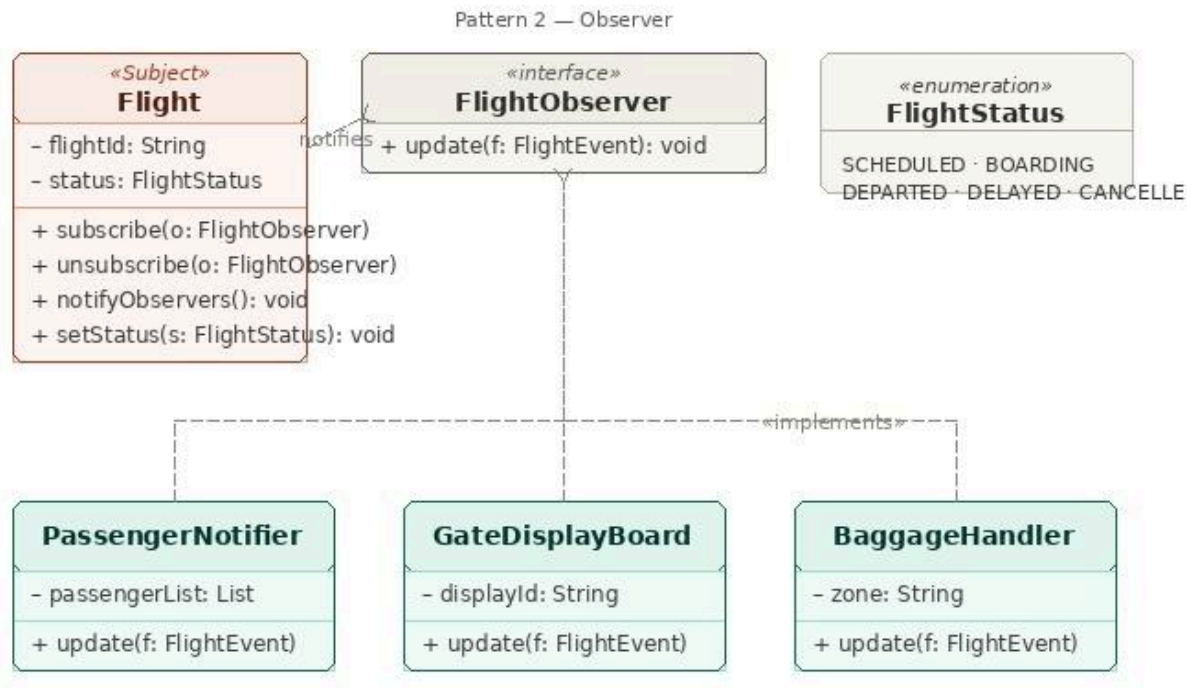
Different check-in methods (online, kiosk, counter) share a common interface but have different behaviors. The Strategy pattern lets the CheckInService swap algorithms at runtime depending on the passenger's chosen method, without branching logic polluting the core service.

Pattern 1 — Singleton



Private constructor prevents direct instantiation; clients always call `getInstance()`

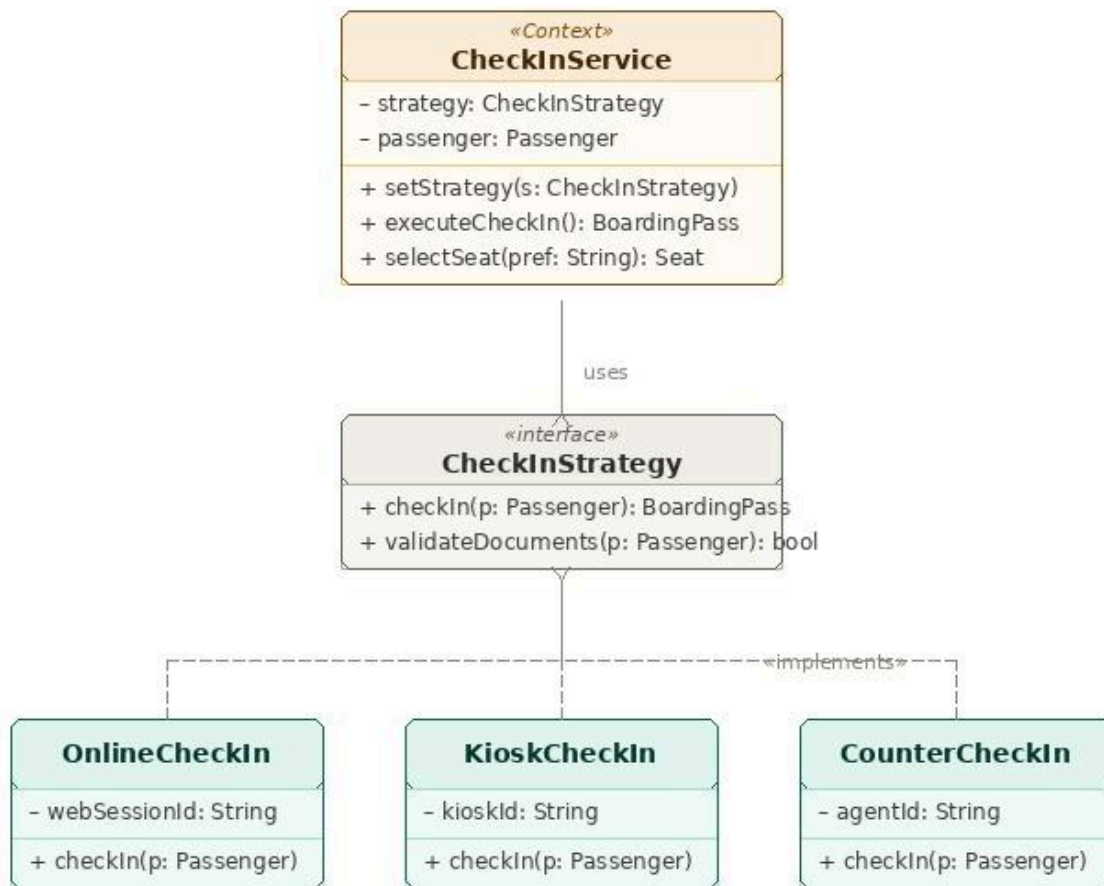
The Singleton diagram shows `AirportControlTower` with a private constructor and a static `getInstance()` method. Both client classes (`FlightDispatcher`, `RunwayMonitor`) access it only through that method — they can never create a second instance.



Any number of observers can be added at runtime without modifying Flight

The Observer diagram shows Flight as the subject holding a list of FlightObserver subscribers. When setStatus() triggers notifyObservers(), all three concrete observers — PassengerNotifier, GateDisplayBoard, and BaggageHandler — receive the FlightEvent through the shared update() interface. New observers (e.g. an airline operations dashboard) can be plugged in without touching Flight.

Pattern 3 — Strategy



The Strategy diagram shows **CheckInService** holding a reference to the **CheckInStrategy** interface. At runtime, `setStrategy()` swaps in whichever concrete strategy is appropriate — **OnlineCheckIn**, **KioskCheckIn**, or **CounterCheckIn** — each implementing `checkIn()` differently without any conditional branching in the context class.